# TOFU: Exploiting and Fuzzing the LLM–Orchestrator–Tool Trust Boundary.


## Abstract

Large language models (LLMs) are increasingly deployed as agents that interact with external software through tools, with an orchestration layer translating model outputs into structured tool invocations. This architecture creates an LLM–orchestrator–tool trust boundary where tool semantics may go unenforced and LLM-generated arguments may be incorrectly treated as trusted. We present TOFU, a tool-aware framework that derives semantic security checks for LLM tools from their existing metadata and a CWE-guided sanitizer corpus. TOFU specializes these checks to individual tools and automatically generates fuzzing harnesses and semantic oracles using runtime introspection and outcome sanitization. We evaluated TOFU on 569 tools across 11 repositories and identified 14 previously unknown bugs corresponding to weakness classes including Arbitrary File Writing, MCP Server Crash, Incorrect Tool Description, and Denial of Service. Of these bugs, seven are input-independent and do not require a specific input to be triggered. Our findings show that real-world tools frequently fail to enforce their documented input semantics, making it unsafe to treat LLM-generated arguments as trusted.


## 1 Introduction

Large language models (LLMs) are increasingly deployed as agents that act on external software through *tools*. These tools allow models to access files, execute programs, query databases, issue network requests, and interact with external services, while orchestration frameworks translate model outputs into structured tool invocations. This transition substantially increases the security consequences of model failures: an unsafe output can now become an operation executed by conventional software with access to sensitive resources resulting in jailbreaks [16], indirect prompt injection [7], tool poisoning [1], tool hijacking [14], and cross-tool attacks [10].

In this paper, we ask a fundamental question: *what level of trust should be established between the LLM and the tool?* We surveyed major orchestration frameworks and protocols by tracing tool registration, argument checking, and dispatch, and found that they largely enforce only structural constraints encoded in schemas, while semantic constraints expressed in natural-language metadata remain unenforced. For example, LangChain and MCP can check that `path` is a string, but not that a path described as "within the workspace" actually remains within that workspace [**?**, **?**]. This creates a gap between the interface presented to the LLM and the guarantees provided to the tool. The gap is security-critical because LLM-generated arguments may still derive from adversarial prompts, retrieved content, or other tools. As we show in Section 4, semantic validation is only one of several missing protections across this boundary. We therefore argue that *LLM-generated tool arguments must remain untrusted at execution time, as tool implementations cannot be assumed to enforce the security properties implied by their documented interfaces*.

Existing techniques are poorly suited to answering this question at scale. Current analyses of LLM tools primarily encode predefined weakness classes, specific information flows, or particular agent–tool attacks [1, 8, 10, 13]. Traditional fuzzing explores broader input spaces, but generally relies on generic oracles such as crashes, memory-safety violations, or assertions. Many tool vulnerabilities produce no such signal. A file tool may successfully access a resource outside its intended workspace, or a network tool may successfully contact a prohibited destination. Detecting these failures requires knowing *what an argument represents* and whether execution respects the corresponding security property.

Our key insight is that LLM tools already expose the semantic information needed to construct such tests. Tool metadata exists precisely to describe to the model what a tool and its arguments mean. Two arguments may both have type `string`, while one represents a filesystem path and another a URL. Their relevant security properties, adversarial inputs, and runtime effects are correspondingly different. We can therefore repurpose existing tool metadata as a source of security specifications for testing the implementation behind the interface.

We present TOFU, a tool-aware framework for automatically discovering semantic security vulnerabilities in LLM tools. TOFU introduces *semantics-aware sanitizers*, a reusable corpus that associates semantic input classes with CWE-guided security properties, adversarial inputs, and the runtime observations needed to identify violations. Given a registered tool, TOFU maps its arguments to applicable sanitizers and specializes those sanitizers using the semantics documented in the tool metadata. An LLM assists with interpreting natural-language metadata, but is not used as the security oracle. TOFU then automatically generates schema-valid fuzzing harnesses and semantic oracles. When implementation-level visibility is available, TOFU uses runtime introspection to observe security-sensitive effects and determine whether they violate the selected sanitizer. For opaque tools, it generates a reference proxy encoding the expected security behavior and uses differential checking to identify inconsistencies between the advertised contract and the target implementation. Thus, rather than asking only whether an execution crashes, TOFU determines whether a tool behaves securely according to the semantics of its own interface.

We evaluated TOFU on 569 tools across 11 open-source repositories using 66 semantics-aware sanitizers. TOFU discovered 14 previously unknown bugs across five repositories, spanning arbitrary file writes, MCP server crashes, denial of service, incorrect tool descriptions, and other semantic security failures. Seven bugs are input-independent and require no particular triggering input. These findings show that securing model behavior and orchestration alone is insufficient: the software ultimately executing LLM-generated actions frequently fails to enforce the security properties its interface advertises. Our contributions are as follows:

- We systematically analyze the LLM–orchestrator–tool trust boundary and identify several missing protections that allow attacker-influenced values to reach privileged tool implementations, establishing why LLM-generated arguments must remain untrusted at execution time.
- We introduce *semantics-aware sanitizers* and design TOFU, which derives tool-specific security properties from existing metadata, generates schema-valid fuzzing harnesses, and detects violations using runtime introspection and black-box differential checking.
- We evaluate TOFU on 569 tools across 11 repositories and discover 14 previously unknown bugs across five repositories, demonstrating that real-world LLM tools frequently fail to enforce the semantic contracts exposed by their interfaces.

# 2 Background

## 2.1 Tool-Enabled LLM Agents and Metadata

Tool-enabled LLM agents augment a language model with external functionality, such as filesystem access, Web retrieval, database queries, and code execution. A typical system consists of an *LLM*, an *orchestration layer*, and a set of *tools*. The orchestrator exposes each tool to the LLM through metadata describing its purpose, arguments, primitive types, and natural-language semantics, then translates model outputs into structured invocations and dispatches them to the corresponding implementations. Importantly, schemas generally capture only syntactic constraints, whereas tool descriptions may specify stronger semantic requirements. For example, a `path` argument may be typed as a string but described as a "file within the workspace," implying a filesystem-containment property absent from the schema. Similarly, strings representing URLs, commands, queries, or hostnames require different security constraints despite sharing the same primitive type. Thus, tool metadata provides a semantic specification of LLM-generated arguments that ultimately become ordinary program inputs at the tool boundary.

## 2.2 Fuzzing Semantic Security Bugs

Traditional fuzzers detect bugs using observable failures such as crashes, assertion violations, sanitizer reports, or abnormal termination. Tool implementations can instead execute successfully while violating their documented semantics. For example, a workspace-local file tool may accept `../../etc/passwd`, successfully access the file, and return normally. We call such failures *semantic security bugs*: executions that satisfy a tool's syntactic interface but violate a security property implied by its documented semantics.

Detecting these bugs requires semantic oracles rather than crash-based ones. The required property depends on how an input is consumed: paths require containment and traversal checks, URLs may require restrictions on schemes and destinations, and command arguments require checks on interpretation by the shell. Consequently, there is no universal sanitizer even for inputs sharing the same primitive type. CWE captures recurring classes of such weaknesses, including path traversal, command injection, improper input validation, and server-side request forgery, but determining which checks apply normally requires program-specific knowledge. Tool-enabled LLM systems expose an unusual opportunity: the argument descriptions already supplied to the LLM encode much of this semantic information, allowing security checks and corresponding fuzzing oracles to be specialized to individual tool arguments.

## 3 Threat Model

ToFu assumes that the orchestration layer faithfully exposes a tool's declared interface and dispatches schema-valid invocations to the corresponding implementation. We trust the ToFu analysis infrastructure, but do not trust the LLM or the tool implementation. In particular, tool metadata describes the behavior expected by the LLM, while the implementation may fail to enforce the corresponding security properties.

We consider an attacker who can influence information consumed by the LLM, including user prompts, retrieved content, and outputs of other tools. Because such information may propagate into structured tool arguments, we treat every schema-valid argument as potentially adversarial. A vulnerability exists when the tool accepts or processes such an argument in a way that violates a security property implied by its documented interface. Vulnerability discovery is independent of a particular LLM or prompting strategy; we separately evaluate whether discovered triggering inputs can be delivered through the normal LLM tool-calling path.

## 4 The LLM–Orchestrator–Tool Boundary

Tool-enabled LLM systems connect language models to conventional software through an orchestration layer. Although frameworks differ in how they organize execution, they share the same basic operation: the LLM produces a structured tool call and the orchestrator translates it into software execution. We examine this interface and identify four security weaknesses that arise from this design.

### 4.1 Generic Tool-Enabled LLM Model

A tool-enabled system consists of an *LLM*, an *orchestrator*, and a set of *tools*. Each tool exposes metadata containing its name, natural-language description, and argument schema. The orchestrator provides this metadata to the LLM, receives a structured invocation

$$C = \langle T, \{a_1 = v_1, \ldots, a_n = v_n\} \rangle,$$

and dispatches the arguments to the corresponding tool implementation. Tool results are returned to the LLM, which may subsequently invoke additional tools.

Frameworks realize this process using different control abstractions. LangGraph, Microsoft Agent Framework, and Google ADK use graph-oriented execution; AutoGen uses message passing; OpenAI Agents SDK and AWS Strands use agent loops; and CrewAI combines task and event-driven workflows. Regardless of the control abstraction, tool execution eventually reduces to the same security-relevant path:

$$\text{Input} \rightarrow \text{LLM} \rightarrow \text{Tool Call} \rightarrow \text{Orchestrator} \rightarrow \text{Tool}.$$

This seemingly simple translation creates four important weaknesses.

*A1: LLMs do not sanitize tool arguments.* Although the LLM produces the final tool call, its arguments may originate from user input, retrieved content, or previous tool outputs. Passing a value through an LLM does not sanitize it. For example, a model can emit `../../etc/passwd` for a string-valued path or preserve shell metacharacters in a command argument. Schema validation establishes that the call is structurally valid, not that its arguments are safe. Thus, LLM-generated arguments remain untrusted when they reach the tool.

*A2: Similar descriptions can hijack tool selection.* The LLM commonly selects tools using their names and natural-language descriptions. Consequently, an attacker can register a tool whose description closely resembles that of a legitimate tool and cause it to compete for the same requests. The same problem applies when an orchestrator first retrieves semantically relevant tools before asking the LLM to select one. ToolHijacker [14] demonstrates such attacks against tool retrieval and selection. Therefore, semantic similarity between tool descriptions cannot establish tool identity or trust.

*A3: Attackers can synthesize tool calls through the LLM.* An attacker who cannot directly access the tool dispatcher may still influence the LLM that produces its input. Given a tool interface, the attacker can specify the desired arguments and induce the model to reproduce the corresponding structured call:

$$\text{Attacker Input} \rightarrow \text{LLM} \rightarrow \text{Tool Call} \rightarrow \text{Tool}.$$

We call this *Repeat-After-Me*. The invocation is syntactically produced by the LLM, but its security-sensitive contents remain attacker controlled. Thus, model generation does not establish that a tool call or its arguments were independently chosen by the model.

*A4: Tool descriptions may not match their implementations.* The metadata consumed by the LLM and the code executed by the tool are separate artifacts. A tool can therefore describe stronger behavior than it actually enforces. For example, a `path` may be described as a "file within the workspace" while the implementation accepts `../../etc/passwd`. Similarly, a tool may describe a URL as remote while allowing loopback destinations. Nothing in the orchestration layer guarantees that the implementation satisfies the semantics advertised in its description.

We reproduced these weaknesses across representative orchestration frameworks (Table 1). Despite their different control-flow models, each ultimately relies on the same LLM-to-tool interface, exposing the same underlying security assumptions.

Together, A1–A4 show that the LLM–orchestrator boundary provides neither input sanitization nor assurance that the

Table 1: Validation of A1–A4 across representative LLM orchestration frameworks. A checkmark denotes a reproduced weakness.

| Framework | Execution Model | A1 | A2 | A3 | A4 |
|---|---|---|---|---|---|
| LangGraph [9] | Graph-based exec. | ✓ | ✓ | ✓ | ✓ |
| MAF [12] | Graph-based exec. | ✓ | ✓ | ✓ | ✓ |
| ADK 2.0 [6] | Graph-based exec. | ✓ | ✓ | ✓ | ✓ |
| AutoGen [11] | Actor Model | ✓ | ✓ | ✓ | ✓ |
| Strands [2] | Model–tool agent loop | ✓ | ✓ | ✓ | ✓ |
| CrewAI [3] | Task/Event Loop | ✓ | ✓ | ✓ | ✓ |

selected and executed tool behaves as advertised. We therefore argue that *LLM-generated tool arguments must remain untrusted at the point of execution*, and that tool implementations must be checked against the security properties implied by their documented interfaces.

## 5 Motivation and Challenges

The weak LLM–orchestrator–tool boundary changes how tool vulnerabilities should be searched for. If the orchestration layer establishes structural validity but not semantic safety, and LLM-generated values may remain attacker-influenced, then a tool argument cannot be trusted merely because it arrived through the normal agent execution path. Consequently, vulnerability discovery need not begin by asking whether a particular attack can induce the LLM to generate a malicious value. Instead, *every schema-valid argument should be treated as potentially adversarial*, and the tool implementation should be tested against the full security contract expressed by its interface. Exploit delivery can then be established separately.

This perspective exposes a limitation of existing approaches. As summarized in Table 2, most begin with a known weakness or attack and search for instances of it. MCP at First Glance [8] applies general-purpose and MCP-specific vulnerability checks; MCP-SecLint [13] tracks tool arguments toward predefined security-sensitive operations; and MCP-Scanner [1] detects a catalog of MCP threats such as tool poisoning and prompt injection. Dynamic approaches improve confidence but retain the same coupling: Chord [10], for example, executes real tools but uses an oracle constructed specifically for cross-tool attacks. These systems are effective for the threats they encode, but their coverage is fundamentally bounded by the vulnerabilities anticipated by the analysis. A semantically invalid filesystem access, network destination, or application-specific value may be security-critical even when it matches no predefined attack pattern or sink.

LLM tools provide a way to search more aggressively. Their metadata already describes what each argument represents and therefore exposes security properties unavailable from its primitive type alone. A `string` described as a "path within the workspace" implies containment; one described as a remote URL implies constraints on schemes and destinations. Rather than starting from a vulnerability and asking which tools match it, we can start from each tool argument, derive the security properties implied by its semantics, and systematically challenge those properties. This reverses the direction of existing analyses and introduces four challenges.

**C1: Deriving Semantic Security Properties.** Security checks cannot be selected from primitive types alone. The same `string` may represent a path, URL, command, or identifier, each with different security requirements. We must therefore infer reusable security properties from the documented meaning of individual arguments and translate them into adversarial input classes and observable violations.

**C2: Specializing Checks to Individual Tools.** A generic weakness does not fully specify correct behavior. Path containment depends on the workspace promised by the tool; network safety depends on its intended destinations and policy. The challenge is to specialize reusable security knowledge to the concrete contract expressed by each tool without requiring a manually written analysis for every implementation.

**C3: Constructing Oracles Under Partial Visibility.** Many semantic violations terminate normally, so crashes provide no useful signal. Moreover, some tool implementations can be inspected while others are available only through their external interface. We therefore need semantic oracles that can establish a violation from execution behavior when visibility is available and from externally observable behavior when it is not.

**C4: Going Beyond a Fixed Vulnerability Corpus.** CWE provides a useful foundation, but it cannot enumerate arbitrary tool-specific policies. Restricting analysis to a fixed set of sanitizers would reproduce the coverage limitation of existing rule-based approaches. The challenge is to specialize or extend reusable security knowledge from tool metadata while keeping the resulting checks grounded in explicit interface semantics rather than unconstrained LLM judgment.

These challenges motivate ToFu: a framework that treats exposed tool arguments as adversarial, derives argument-specific security properties from their documented semantics, and automatically constructs the inputs and oracles needed to test them. We separately evaluate whether discovered bugs can be reached through normal LLM tool-calling paths.

## 6 Overview

ToFu is a tool-aware fuzzing framework that turns the semantic information already exposed by LLM tools into executable security tests. As shown in Figure 1, ToFu takes a registered tool as input and automatically derives three artifacts:

Table 2: Comparison with published approaches for automatically identifying security weaknesses in LLM tools and MCP servers. Existing approaches either target predefined vulnerability classes or specific agent–tool attacks. TOFU instead derives argument-specific security properties from tool metadata and uses them to dynamically discover semantic implementation bugs.

| System | Tool Impl. | Dynamic | Argument Semantics | Broad Bug Classes | Semantic Oracle | Auto. Harness |
|---|---|---|---|---|---|---|
| MCP at First Glance [8] | ● | ○ | ○ | ◐ | ○ | N/A |
| MCP-SecLint [13] | ● | ○ | ○ | ○ | ○ | N/A |
| MCP-Scanner [1] | ◐ | ● | ○ | ◐ | ○ | ● |
| Chord [10] | ○ | ● | ◐ | ○ | ◐ | ● |
| **TOFU** | ● | ● | ● | ● | ● | ● |

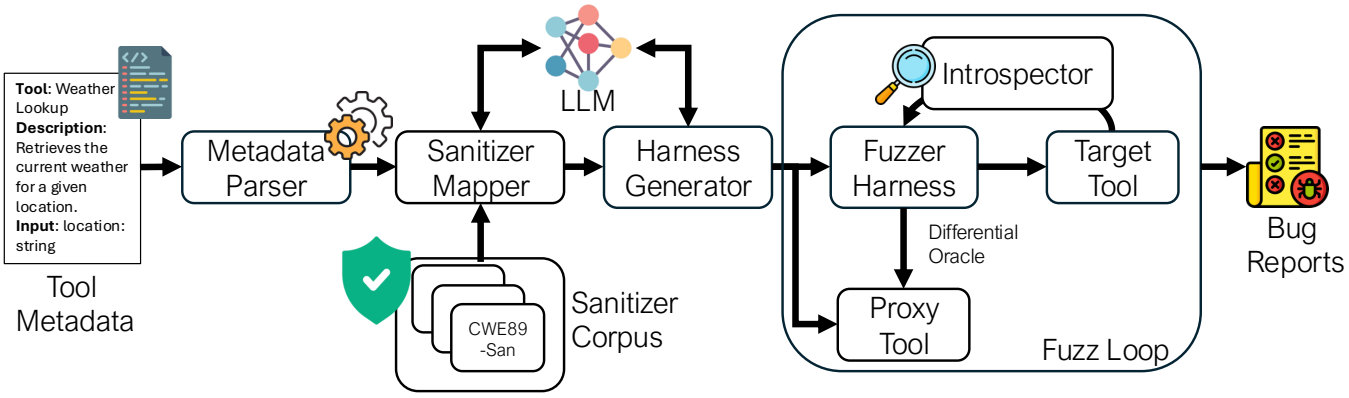



Figure 1: TOFU maps tool metadata to semantics-aware sanitizers, generates schema-valid fuzzing harnesses and, when needed, a proxy oracle, and detects semantic security violations through runtime introspection or differential checking.

the security properties to test, a harness that exercises those properties, and an oracle that determines whether they are violated. TOFU operates through the tool's existing orchestration interface, reusing its metadata discovery and invocation mechanisms without requiring changes to the LLM or target tool.

The **Metadata Parser** first normalizes the tool description, argument schemas, and natural-language argument descriptions. The **Sanitizer Mapper** uses this information to associate each argument with applicable checks from a CWE-guided **Sanitizer Corpus**. Each sanitizer couples a semantic input class with adversarial inputs, an expected security property, and the observations needed to establish a violation. For example, a path argument can induce containment and traversal checks, while a URL can induce scheme and destination checks. An LLM assists in mapping and specializing these generic sanitizers to the semantics documented by the tool, but the resulting sanitizer, rather than the LLM, defines the security oracle.

The specialized security specification drives the **Harness Generator**. It constructs a **Fuzzer Harness** that mutates arguments while preserving the target's schema, allowing adversarial values to traverse the normal orchestration path and reach the implementation. For targets where internal behavior cannot be observed, the generator additionally constructs a **Proxy Tool** that encodes the expected behavior of the specialized security checks. Thus, the same semantic specification determines both what inputs are generated and what behavior is considered secure.

Finally, the **Fuzz Loop** executes generated invocations against the **Target Tool**. When implementation-level visibility is available, the **Introspector** observes sanitizer-relevant effects, such as filesystem, process, or network operations, and directly checks the corresponding property. Otherwise, the **Differential Oracle** compares the target against the generated proxy and identifies cases where the target accepts or processes an input that violates the inferred contract. Crashes and unhandled failures provide additional feedback. When a violation is detected, TOFU reports the triggering invocation, affected argument, expected property, and observed behavior. Section 7 describes each stage in detail.

## 7 Design

TOFU takes a registered tool and constructs a coverage-guided fuzzing target with automatically selected security checks from the information already exposed through its orchestration interface. The design has four stages (Figure 1): (1) normalize the tool interface, (2) select security sanitizers for each tool, (3) generate schema-valid fuzzing inputs and the corresponding semantic oracles, and (4) execute the target using

conventional and semantic feedback. Tool metadata informs both seed generation and sanitizer selection, ensuring that the property being exercised is also the property used to judge the execution.

## 7.1 Metadata Extraction

TOFU begins from the metadata already exposed to the LLM. For a tool $T$, the Metadata Parser constructs

$$T = \langle D_T, \{a_1, \ldots, a_n\} \rangle,$$

where $D_T$ is the tool-level description and each argument $a_i$ denotes an entry in the available input schema. The implementation retains the raw schema for use by the LLM and the input decoder, including argument names and types. The tool's natural-language description is additionally provided to the LLM for selecting sanitizers and generating seeds.

The parser obtains this information through the tool registry rather than requiring a separate fuzzing specification. This is important because the resulting representation captures exactly the interface presented to the LLM and therefore the semantic contract that TOFU later tests.

To handle the structural heterogeneity of tool interfaces across different ecosystems (e.g., MCP JSON-RPC declarations, LangChain and AutoGen Pydantic models, and FastMCP/Python type annotations), the Metadata Parser normalizes disparate definitions into a canonical, JSON-Schema-aligned representation. Primitive types (`string`, `integer`, `number`, `boolean`) and argument-level docstrings are extracted from schema fields or introspected signatures. For compound structures, such as nested objects and arrays, the parser recursively traverses sub-schemas (`properties` and `items`), preserving hierarchical access paths while exposing leaf-level attributes as distinct argument descriptors $a_i$ with their own natural-language contexts. Polymorphic parameters specified via unions (`oneOf`, `anyOf`, or Python `Union`) are preserved as multi-type disjunctions, allowing subsequent stages to specialize sanitizers and generate test cases for each candidate branch. Discrete value domains (`enum` and `Literal`) are retained as explicit finite sets to constrain baseline generation. Finally, the parser partitions arguments into required and optional sets based on schema constraints and the presence of default values $v_{\text{def}}$, enabling the harness generator to systematically alternate between partial invocations relying on defaults and fully specified adversarial payloads.

## 7.2 Semantics-Aware Sanitizers

The central abstraction in TOFU is a *semantics-aware sanitizer*. Traditional sanitizers associate a program execution with a largely application-independent property, such as whether a memory access is in bounds. Tool vulnerabilities instead depend on what an input represents. A string interpreted as a path requires different security checks from the same string interpreted as a URL or command.

We represent a sanitizer as

$$S = \langle A, G, O, I \rangle,$$

where $A$ is an *applicability specification*, $G$ describes adversarial input classes, $O$ defines the expected security property, and $I$ describes the execution observations required to evaluate that property. The four components deliberately connect input generation and bug detection. For example, a filesystem-containment sanitizer may apply to path-like arguments, generate absolute paths and traversal sequences, require that resolved accesses remain beneath an allowed root, and request observations of filesystem operations. A URL sanitizer instead generates adversarial schemes and destinations and evaluates the resulting network behavior.

The initial Sanitizer Corpus organizes these specifications around recurring CWE weakness classes. CWE is used as a source of reusable security knowledge, not as the oracle itself: each corpus entry must translate a weakness into concrete applicability conditions, input classes, expected behavior, and observable evidence. Table 7 lists the CWE-based sanitizers used for TOFU.

In TOFU, corpus entries are implemented as hybrid specifications combining declarative metadata for LLM interpretation with executable Python components for fuzzing and monitoring. Specifically, **Applicability ($A$)** is a structured JSON object with natural-language docstrings and semantic tags (e.g., associating CWE-22 with keywords like `path` or `directory`) consumed by the LLM during mapping. **Adversarial Generation ($G$)** provides structured JSON payload suites (e.g., `../../etc/passwd`, encoded escapes, and absolute paths) to guide mutation. **Security Oracle ($O$)** is an executable Python predicate (e.g., `check_introspection_path_traversal`) evaluated against a behavioral rule (`on_trigger: raise_exception`), verifying that resolved filesystem accesses remain within an authorized root directory. **Execution Instrumentation ($I$)** comprises executable context-manager tracers (e.g., `FileAccessTracer` intercepting `builtins.open`) that collect concrete operations into the execution trace $\tau(T, x, I_S)$.

## 7.3 Mapping Sanitizers

For each argument $a_i$, TOFU identifies the sanitizers whose applicability conditions match its documented semantics:

$$\mathcal{S}(a_i) = \{S_j \mid S_j \text{ applies to } a_i\}.$$

This decision cannot generally be made from the primitive type alone. TOFU therefore considers the argument name, argument description, schema, and surrounding tool description.

An LLM interprets this natural-language context and selects candidate sanitizers from the corpus.

The LLM does not decide whether an execution is vulnerable. Its role is restricted to connecting an explicitly defined security property to an argument whose semantics are expressed in natural language. The resulting security decision remains grounded in the sanitizer's oracle.

To select the sanitizers used for a fuzzing campaign, the LLM is provided with the tool name, description, arguments, and the list of available sanitizers. The LLM then makes a selection based on which sanitizers are applicable to the likely classes of vulnerabilities for the tool.

If the LLM hallucinates by listing a non-existing sanitizer, TOFU detects it and simply does not add such sanitizer to the list. However, the LLM does not choose the expected sanitizers it will degrade the performance of TOFU, thus the choice of sanitizers made by the LLM has to be monitored to make sure results are accurate.

## 7.4 Schema-Aware Harness Generation

Once TOFU has instantiated the applicable security properties, it constructs a fuzzing harness for the tool. The harness operates on structured arguments rather than on the serialized tool request. This distinction is important: arbitrary byte-level mutations would frequently violate the invocation schema and be rejected before reaching the implementation.

For each execution, the harness constructs a schema-valid invocation

$$C = \langle T, \{a_1 = v_1, \ldots, a_n = v_n\} \rangle$$

and selects one or more arguments for semantic mutation. Values for the selected arguments are generated using the corresponding sanitizer's adversarial input classes, while the remaining arguments are populated with values satisfying their declared schemas. A path sanitizer, for example, can vary the path among traversal, absolute-path, normalization, and encoding cases without corrupting the surrounding tool invocation.

This design separates *structural validity* from *semantic adversariality*: the orchestrator should accept the invocation structurally, allowing the test to exercise whether the tool itself enforces the stronger security property.

The LLM is provided the target tool's description and arguments to construct a set of initial seeds following the tool's schema. With the initial corpus created, the fuzzer then follows a typical fuzzing campaign using Atheris and its built in mutation engine. The feedback mechanism uses coverage to capture interesting inputs, and captures crashing inputs either from sanitizer output or from actual crashes.

The seed generation is exclusively based on the LLM knowledge combined with the tool's basic information. The assumption is made that the tool is runnable in the current environment.

## 7.5 Semantic Oracles

A schema-valid malicious input is useful only if TOFU can determine whether the resulting execution violates its intended property. Crashes, exceptions, and conventional sanitizer failures remain useful signals, but many target bugs terminate normally. TOFU therefore derives a semantic oracle from the same specialized sanitizer used to generate the input.

TOFU evaluates the property through both runtime introspection and outcome sanitization.

### 7.5.1 Runtime Introspection

When the target execution can be observed, the sanitizer specifies which security-sensitive events are relevant to its oracle. For example, filesystem-containment checks require the paths actually accessed by the target, whereas SSRF checks require the destination actually contacted. The Introspector collects only those event classes required by the active sanitizer.

Let $\tau(T, x, I_S)$ denote the execution observations collected for target $T$, input $x$, and sanitizer-required instrumentation $I_S$. The specialized oracle evaluates

$$O_S^T(\tau(T,x,I_S)) = \begin{cases} 1 & \text{if the specialized property is violated,} \\ 0 & \text{otherwise.} \end{cases}$$

For containment, for example, the oracle can canonicalize an observed path and test whether it escapes the specialized root. Thus, an execution that successfully reads an unintended file becomes a fuzzing failure even though the tool returns normally.

Each sanitizer has a mapping to introspection events from Python's introspection library to analyze the tool's behavior at runtime. The majority of these events are for Python runtime API hooking, enabling the fuzzer to observe file operations, system calls, and other interesting API usage based on the sanitizer.

Upon triggering an interesting event, the introspection hook adds details of the event to an internal list. The fuzzer then runs the corresponding sanitizers on the recorded events to check if an event is flagged as a violation. An input flagged by one of the sanitizers is then recorded and fuzzing continues.

Specifically, filesystem introspection hooks intercept raw file paths at the runtime API boundary (e.g., `open`) before path resolution occurs. The corresponding containment oracle then canonicalizes both the accessed path and the authorized base directory into their absolute, canonical forms (resolving relative traversals such as `../`, redundant separators, and symbolic links). By computing the relative path from the designated workspace root to the canonical target, the oracle accurately detects directory escapes and symlink traversals regardless of path encoding or aliasing tricks used by the fuzzed input.

| Hook | Monitored Property |
|---|---|
| `FileAccess` | File access & directory escape |
| `FileWrite` | Dangerous file creation & upload |
| `TempFileLeak` | Uncleaned temporary files |
| `CommandExecution` | Command & CLI flag injection |
| `CodeExecution` | Dynamic code evaluation |
| `Deserialization` | Unsafe object deserialization |
| `SQLExecution` | Raw / unparameterized SQL queries |
| `NetworkExecution` | Outbound requests & SSRF |
| `SSLCertValidation` | Disabled TLS certificate checks |
| `UnrestrictedIPBinding` | Wildcard interface binding |
| `UnverifiedCodeDownload` | Code download without signature |
| `BlockingEventLoopCall` | Blocking calls in async loop |
| `ReDoS` | Catastrophic regex backtracking |
| `DataAmplification` | Decompression & Zip bombs |
| `LoopCondition` | Unchecked loops & CPU exhaustion |
| `NonceReuse` | Cryptographic Nonce/IV reuse |
| `WeakPRNG` | Insecure random generation |
| `WeakHash` | Broken hash algorithms (MD5/SHA1) |
| `SensitiveDataLogging` | Plaintext secrets in logs |
| `LogInjection` | CRLF / log forging sequences |
| `XXE` | XML entity expansion |

Table 3: Introspection hooks and monitored properties.

#### 7.5.2 Outcome Sanitization

In addition to introspection, TOFU uses selected sanitizers as *contract-based outcome oracles*. A sanitizer evaluates the fuzzed input $x$ against a security policy and, when $x$ violates that policy, checks whether the target produced an appropriate safe outcome.

Let $V(x)$ denote that the sanitizer considers $x$ invalid or malicious, and let $Reject(T(x), E)$ denote that the target rejected $x$, where $E$ may include observed runtime side effects when available. The primary discrepancy of interest is

$$V(x) \quad \wedge \quad \neg Reject(T(x), E).$$

The oracle therefore answers a narrow question: according to the security contract derived from the tool interface, should this invocation have been accepted? The oracle may also treat network, process, or file-system side effects as evidence to whether an input was accepted or not. However, the most common check is a simpler input to output check for whether a target accepts a malicious input without raising an error or giving a safe fallback.

Nondeterminism (e.g., dynamic timestamps or network latency) is mitigated by evaluating semantic invariants—such as structured error types and schema rejections—rather than brittle byte-level outputs. For tools with absent side effects (e.g., pure in-memory string or JSON transforms), the oracle relies on output validity predicates and execution health (catching unhandled exceptions or algorithmic stalls like ReDoS). For tools with ambiguous or silent outcomes, the oracle cross-references the tool's return status with runtime introspection traces (e.g., matching logged internal warnings or unperformed sink operations against reported success) to disambiguate whether the input was truly accepted.

| Component | Modules & Descriptions | SLOC |
|---|---|---|
| Sanitizers & Oracles | `src/sanitizers/` (21 CWE introspection hooks & checks) | 3,580 |
| Target Runners | `src/runners/` (In-process Python & Node.js/MCP runners) | 2,700 |
| Fuzzing Engine | `src/fuzzer.py`, `src/input_parser.py` (Schema-aware mutation) | 1,650 |
| Support & Telemetry | `src/metrics.py`, `src/reporters.py`, `src/llm.py` (Seeding & logs) | 470 |
| **Total Core Code** | *Excludes tests, prompts, and third-party tools* | **8,400** |

Table 4: Codebase breakdown by major component.

### 7.6 Fuzzing Loop and Feedback

The fuzzing loop processes tools sequentially. For each tool, a list of appropriate sanitizers are chosen based on the tool's metadata, an initial seed corpus is generated, and the coverage-guided fuzzing engine is invoked. The engine continuously mutates inputs, which are mapped to the schema and executed by the fuzzer. Each execution is then evaluated for expanded coverage, a triggered crash, or sanitizer output.

An execution is interesting when it increases the measured coverage, guiding the input mutation toward deeper program states, while the oracle is responsible for determining if those states violate the security contract. Crash and sanitizer output classify an input as a potential failure or policy violation.

When TOFU detects a violation, the fuzzer records the triggering invocation together with the sanitizer, expected property, and evidence that caused the oracle to fire. These are outputs of the fuzzing process rather than a separate architectural component.

## 8 Implementation

We implemented TOFU as an 8,400-SLOC Python framework (Table 4, excluding tests and libraries), running on Python 3.12, Atheris v2.3, LangChain Core v0.3, the official `mcp` SDK, and Node.js v20+.

**Sanitizer Specialization & LLM Seeding.** TOFU prompts MiniMax M3 ($T = 0.7$, up to 4 retries, fixed prompt in Appendix **??**) with the target tool's metadata, JSON schema, and attack vectors from an internal seed bank to produce $N =$ 6–12 structured seeds. To synthesize valid binary documents, TOFU exposes native Python tool bindings (`reportlab` for PDF, `python-docx` for DOCX, `openpyxl` for XLSX, `zipfile` for archives); the LLM invokes these tools via standard function calling, materializing files to disk and referencing them as `predefined_seed:<path>`. A deterministic fallback populates boundary numbers $(0, -1)$, boolean toggles, and malformed types if the LLM fails ($< 2\%$ of queries). Concurrently, TOFU matches schema properties against a 21-CWE corpus (`src/sanitizers/`), compiling executable Python verification predicates cached per schema hash.

**Fuzzing Engine & Runtime Interception.** TOFU recursively decodes libFuzzer-mutated

byte streams into schema-valid arguments using `atheris.FuzzedDataProvider` (`FuzzInputParser`), mapping strings (`ConsumeUnicodeNoSurrogates`), numbers, arrays, objects, and deliberate type-mismatches. Feedback is measured as unique Python bytecode branch transitions via `sys.settrace` (and Atheris C-hooks), strictly filtering out framework internals (`src/fuzzer.py`, `src/sanitizers/`). To extract libFuzzer feature updates without disk I/O or terminal contention, C-level `stderr` is duplicated (`dup2`) into an IPC pipe drained asynchronously by a worker thread. Dual oracles evaluate each run: (1) *Runtime Introspection*, where `ExitStack` wraps execution sinks (`open`, `subprocess`, `socket`, `asyncio.get_running_loop`) to evaluate path containment, argument injection, and async loop freezing; and (2) *Black-Box Differential*, checking schema contracts and silent failures (`failed_init_non_exit`).

**Target Execution & Telemetry.** ToFU supports in-process execution for Python tools (`PythonRunner`) and stdio subprocess workers for JS/TS MCP servers (`JSRunner`). Each iteration runs under a 5-second watchdog in an isolated temporary directory (`mkdtemp`) with environment variable restoration. Crashes are deduplicated by hashing the top-3 normalized target stack frames grouped by triggered sanitizer ID. Every trial logs execution latency, input values, triggered sanitizers, return status, edge coverage, and crash signatures.

# 9 Evaluation

TODO [Pierre: Add explicit evaluation questions at the beginning. Right now the evaluation reports results without clearly connecting each subsection to the main claims made by ToFU. The questions should cover the applicability of the sanitizer corpus, semantic bug discovery, runtime filtering, fuzzing effectiveness, and real-world vulnerability discovery. You can phrase these based on the experiments that are already available.]

We structure our evaluation around the following research questions: RQ1 - Sanitizer applicability. To what extend can ToFU's sanitizer corpus be mapped to the semantics exposed by real-world LLM tools?

RQ2 - Semantic bug detection. Can ToFU detect violations of tool-specific semantic contracts that would not necessarily manifest as conventional crashes or schema violations?

RQ3 - Runtime filtering. Can ToFU distinguish security-relevant semantic violations from benign runtime behavior ovserved during normal tool execution?

RQ4 - Fuzzing effectiveness.

TODO [Pierre: Add an experimental setup subsection using the information already available. Report the machine configuration, operating system, Python and Node.js versions, number of cores used for each run, timeout policy, and fuzzing budget. Also state whether each reported result comes from a single run or multiple runs so that the methodology is explicit.]

## 9.1 Tool selection

We evaluated our approach on 569 tools collected from 11 open-source repositories. The repositories were chosen by taking the top MCP servers listed on https://github.com/mcp that could be fuzzed locally with simple setup. In addition, we've taken the first result when searching for "AI tool" on GitHub.

TODO [Pierre: The repository selection methodology needs to be reproducible. Define exactly what top MCP servers means, give the date on which this ranking was collected, report the initial candidate set if that information is available, and state the inclusion and exclusion criteria. In particular, define what could be fuzzed locally with simple setup means.]

TODO [Pierre: The first result when searching for AI tool on GitHub is difficult to defend as a systematic sampling criterion. If this repository was included for a specific reason, explain that reason and treat it as an intentionally selected target rather than presenting the GitHub search result as part of a systematic sampling methodology.]

Table 5 details the complete breakdown of the 11 target repositories, spanning both native Python tools and Node.js MCP servers.

TODO [Pierre: The dataset is heavily dominated by tools-main, which contributes 412 of the 569 tools. Acknowledge this explicitly and avoid using 569 tools as evidence of 569 independent projects or ecosystems. If the information is already available, also report the Python versus JavaScript and TypeScript breakdown and the major categories represented in the dataset.]

TODO [Pierre: Add the exact repository revisions or commit hashes used in the evaluation. If some candidate tools from these repositories could not be evaluated, state how many were excluded and why. This is important for reproducibility.]

The tools used for evaluation were then all publicly-available tools, most of them being in projects with more than 1,000 GitHub stars.

TODO [Pierre: Quantify most by giving the exact number of repositories or tools above this threshold. Also briefly explain why these repositories provide a useful sample of real LLM tools. GitHub stars can support evidence of visibility or adoption, but should not be used alone to claim representativeness.]

The analysis used 66 sanitizers in total: 45 CWE-based sanitizers, 6 web-specific sanitizers, 5 MarkItDown-specific sanitizers, 5 HAR/ZIP-specific sanitizers, and 5 generic sanitizers.

TODO [Pierre: This needs a careful methodology explanation because a reviewer may suspect target-specific overfitting. State when these 66 sanitizers were created and whether any sanitizer was added or modified after observing behavior or bugs in the evaluated targets. This is especially important for the MarkItDown-specific and HAR or ZIP-specific sanitiz-

Table 5: Breakdown of the 11 evaluated target repositories and tools.

| Repository | Runtime | Description / Category | Tools |
|---|---|---|---|
| `tools-main` | Python | Standalone Agent Tool Suite | 412 |
| `pmb` | Python | Memory & Cognitive Graph MCP | 68 |
| `serena` | Python | Workspace & Coding Agent MCP | 43 |
| `memo` | Python | Knowledge Graph & Memory MCP | 17 |
| `basic-memory` | Python | FastMCP Semantic Note Storage | 9 |
| `markitdown` | Python | Document & Media Converter | 1 |
| `chroma-mcp` | Python | Vector Database Retrieval MCP | 3 |
| `terravision` | Python/JS | Geospatial Visualization MCP | 4 |
| `chrome-devtools-mcp` | Node.js/TS | Browser Automation MCP | 6 |
| `DesktopCommanderMCP` | Node.js/TS | OS & Terminal Control MCP | 4 |
| `brightdata-mcp` | Node.js/TS | Web Scraping & Proxy MCP | 2 |

ers because these formats also appear among the discovered vulnerabilities.]

TODO [Pierre: Explain how the different sanitizer groups were obtained. In particular, distinguish sanitizers directly derived from CWEs, generic sanitizers, and format-specific sanitizers. The reader should understand whether the format-specific sanitizers are manually authored, generated from metadata, or specialized automatically by ToFU.]

To seed the fuzzing process across these tools, TOFU employs two complementary input generators: (1) an LLM-guided generator equipped with 8 native binary document builders (`create_pdf`, `create_docx`, `create_xlsx`, `create_pptx`, `create_zip`, `create_xml`, `create_html`, `create_json`) and a categorized attack Seed Bank, and (2) a deterministic fallback mapper generating boundary numbers $(0, -1, 2^{31}-1, -2^{31}, \pm 10^{308})$, boolean permutations, empty structures, and type confusion payloads.

TODO [Pierre: Add the corpus statistics that are already available. Useful information includes the number of initial seeds generated per tool, the number of tools requiring document builders, whether the LLM generator failed for any targets, and how often the deterministic fallback was used. If some of these statistics were not recorded, leave them for later rather than estimating them.]

## 9.2 Found vulnerabilities

Overall, the evaluation uncovered 14 vulnerabilities across five repositories. These vulnerabilities span several distinct classes of security-relevant behavior.

TODO [Pierre: If these numbers are already available, report the funnel from ToFU observations to the final 14 findings. For example, state how many raw alerts were generated, how many unique candidates remained after deduplication, and how many were manually confirmed. This is useful for understanding the amount of manual triage. Do not add numbers that were not recorded.]

TODO [Pierre: Add the current disclosure and validation status for each finding. Indicate whether each issue has been reported, acknowledged, fixed, or assigned a public issue, advisory, or CVE where applicable. Maintainer confirmation will strengthen the vulnerability claims.]

First, three vulnerabilities involve unintended access to local resources through interfaces that are expected to manipulate remote resources. In particular, tools accepting URLs may also accept `file://` URIs, allowing local files to be opened or rendered.

A second category consists of unsafe filesystem operations. Two vulnerabilities allow attacker-controlled input to influence where files are written: one through an insufficiently constrained output path and another through path traversal during HAR/ZIP extraction.

Four vulnerabilities affect availability and robustness. Malformed or unexpected inputs can terminate the tool or MCP server through `sys.exit`, uncaught exceptions, invalid filesystem strings, or failed assertions. Because MCP servers may expose several tools through a single long-running process, such failures can affect more than the individual tool invocation that triggered them.

Two additional vulnerabilities concern trust-boundary and initialization failures: one component communicates with another process without requiring authentication, while another fails open when initialization of its language-server dependency fails.

The remaining three vulnerabilities concern resource and interface hygiene: a temporary file is not removed after use, attacker-controlled strings are written to logs without escaping, and one tool relies on a deprecated MCP SDK annotation.

TODO [Pierre: Establish the security impact of each category under the paper's threat model. State who controls the triggering input, what privilege the tool or MCP server has, and what security property is violated. This is especially important for the temporary-file cleanup issue, log injection, and deprecated API finding because reviewers may otherwise classify these as reliability or software-quality issues rather than security vulnerabilities.]

We classify the 14 identified issues into two categories:

- Semantic Input Validation Vulnerabilities (9 issues), which require specialized argument-level semantic payloads and runtime introspection oracles to be discovered
- Architectural and Lifecycle Defects (5 issues), which manifest as process-level design flaws, improper error recovery, or missing IPC authentication independent of specific tool argument payloads

TODO [Pierre: Explain exactly how the five Architectural and Lifecycle Defects were discovered. Because the text says that these defects are independent of specific tool argument payloads, the reader needs to know whether they were automatically detected by initialization or runtime telemetry, surfaced while investigating another ToFU alert, or found through manual analysis. Be precise about what ToFU itself discovered.]

To isolate these 14 genuine flaws from benign runtime activity, TOFU pairs each sanitizer with an explicit behavioral rule (`raise_exception`, `allow_success_with_fallback`, or `allow_success_always`) and workspace boundary constraints. This semantic filtering suppresses normal telemetry noise (e.g., reading configuration files like `.env` or executing `git rev-parse HEAD` during framework startup) while flagging contract violations.

TODO [Pierre: Explain how these behavioral rules are assigned to sanitizers and give enough information to understand why the examples listed here are considered benign. If the counts are already available, report how much filtering occurs before the final alerts are produced.]

The 14 found vulnerabilities are listed in table 6. The most critical ones allow to crash the MCP server, to access the MCP server local files, to overwrite its existing files or to saturate its storage space, and to store files outside of intended directory when decompressing a compressed archive.

TODO [Pierre: Expand this table if the relevant information is already available, or add a second table, to better connect each finding to how ToFU detects it. Useful fields include the violated property or CWE, triggering condition, sanitizer or oracle responsible for detection, whether a fuzzing input is required, and disclosure status. The important connection is from ToFU mechanism to triggering input or condition to observed behavior to confirmed vulnerability.]

TODO [Pierre: Explicitly state that the 14 entries correspond to distinct root causes and briefly explain the deduplication methodology. Several findings occur in the same component, so the reader needs to understand why they are counted as separate vulnerabilities.]

TODO [Pierre: Add a short quantitative summary using information that is already available, such as the number of affected repositories, affected tools, semantic findings, input-independent findings, and findings acknowledged or fixed by maintainers.]

## 9.3 Code coverage

Figure 2 shows the proportion of code covered after 24-hour runs for several tools. Depending on the amount of total features in the tool, the coverage has either reached its final coverage almost instantly, or has taken hours to reach its final number. In both cases, most of the coverage for the tool features was reached in less than an hour. Code coverage is strictly scoped to the first-party source code of each target tool/repository, explicitly excluding third-party dependencies, standard runtime libraries, and test suites.

TODO [Pierre: Explain why these particular tools were selected for the coverage figure. State whether they were chosen as representative examples, because coverage instrumentation was available, or according to another criterion. This is important because the evaluation contains 569 tools but the figure reports only a small subset.]

TODO [Pierre: Define the coverage metric and denominator precisely. The current discussion refers to features, branch coverage, and basic blocks. These terms should not be used interchangeably. Explain exactly what is measured, how the total first-party code is calculated, and what the basic-block number shown in each bar represents.]

TODO [Pierre: The text states that most coverage was reached in less than one hour, but the current figure reports only the final 24-hour result. If you already have the time-series data described in the Implementation section, either show it or avoid making a stronger temporal claim than the presented figure supports.]

TODO [Pierre: Be careful when describing Python and JavaScript coverage. If the two runners use different coverage definitions, explain the difference and avoid presenting their percentages as directly comparable.]

Across these targets, TOFU operates with two execution runners:

- **In-Process Python Runner (`PythonRunner`):** Dynamically invokes Python functions directly within the fuzzer process, enabling over 20 low-overhead introspection context-manager tracers (intercepting `open`, `subprocess.Popen`, `eval`, etc.).
- **Socket-Driven Node.js Runner (`JSRunner`):** Executes JavaScript/TypeScript MCP servers as child processes communicating over Unix domain sockets (`coverage_wrapper.js`). It collects V8 branch coverage at runtime and forwards it to the Atheris engine via an in-memory trampoline (`fake_coverage.py`).

TODO [Pierre: Most of the runner mechanism belongs in the Implementation section. In this section, retain only the details necessary to explain how coverage is collected for the two runtime environments.]

Table 6: Summary of identified issues.

| Project | Component | Class | Semantic |
|---|---|---|---|
| tools | `http_check` | Local file access | Yes |
| markitdown | `convert_to_markdown` | Local file access | No |
| tools | `extract_issues` | Unhandled process termination | Yes |
| chrome-devtools-mcp | `navigate_page` | Local resource access | No |
| tools | `extract_issue_numbers` | Uncontrolled output path | Yes |
| tools | `extract_issue_numbers` | Improper input validation | Yes |
| tools | `extract_har` | Path traversal | Yes |
| tools | `extract_har` | Unhandled malformed archive | Yes |
| serena | Task executor | Resource cleanup issue | No |
| serena | Task executor | Log injection | No |
| serena | `FindSymbolTool` | Invalid input handling | Yes |
| serena | JetBrains client | Missing authentication | No |
| serena | Project query tools | Fail-open initialization | No |
| memo | `memo_version` | API compatibility issue | No |

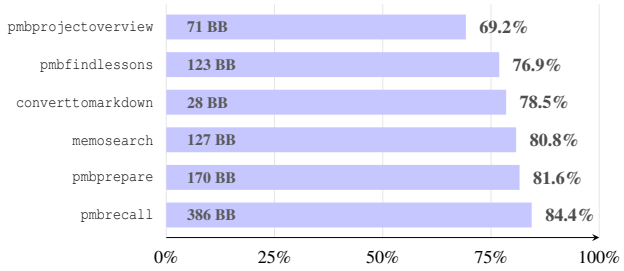



Figure 2: Code coverage after 24 hours.

## 10 Case Study

TODO [Pierre: The current section explains the vulnerability well, but it does not yet function as a case study of ToFU. The key purpose of this section should be to walk through the complete discovery process. Explain what metadata ToFU receives for this tool, which sanitizer is selected and why, what input or seed is generated, how fuzzing transforms that input into the triggering argument, what runtime operation the introspection layer observes, and why the semantic oracle identifies that operation as a violation. The reader should be able to follow the bug from tool metadata all the way to the final vulnerability.]

TODO [Pierre: At the beginning of the case study, briefly explain why this particular bug is representative of ToFU's contribution. The interesting point is not simply that command-line argument injection exists, but that the argument is syntactically a valid string and is passed safely without a shell, while its semantic interpretation by the downstream command violates the intended tool contract. This is exactly the distinction the paper should emphasize.]

An example of a vulnerability found using ToFU that can be studied can be found in `tools-main/python/extract_issues.py`.

TODO [Pierre: Before the code listing, describe the tool's documented interface. State what range_spec is supposed to represent according to the tool metadata or description and, if available, include the relevant description supplied to ToFU. This establishes the intended semantic contract before showing how the implementation violates it.]

```
def extract_issue_numbers(range_spec: str) -> str:
    ...
    git_range = range_spec if ".." in range_spec
    else f"{range_spec}..HEAD"
    ...
    result = subprocess.run(
        ["git", "log", git_range, "--pretty=%B"],
        capture_output=True,
        text=True,
        check=True,
    )
    ...
```

TODO [Pierre: Immediately after the listing, connect this code to ToFU rather than moving directly to manual vulnerability analysis. Explain which sanitizer ToFU associates with range_spec and why the tool metadata makes that sanitizer applicable. If ToFU specializes a generic CWE-88 sanitizer to this argument, show the resulting check or summarize it here. This is the most important missing step in the current case study.]

In this case, range_spec is expected to be a string representing either a single tag/commit or an explicit Git range.

TODO [Pierre: Explain how this expectation is known. Ideally tie it directly to the tool description, argument metadata, documentation, or generated sanitizer. This is important because ToFU's claim is that the security property follows from documented tool semantics rather than a manually written specification.]

By default, `subprocess.run` is invoked with a list of arguments and without a shell interpreter (`shell=False`);

therefore, classic command chaining and shell injection (e.g., providing `range_spec="; echo success"` or piping operators) are inherently prevented.

TODO [Pierre: This distinction is important and should be connected explicitly to the motivation for semantic sanitization. A conventional command-injection check looking for shell metacharacters would consider this invocation substantially safer because shell interpretation is disabled. ToFU instead needs to reason about the semantics of the argument accepted by Git itself. Make that point explicit.]

However, passing unsanitized arguments directly into a command invocation opens the door to *CLI Argument Injection* (CWE-88). Because the argument is passed as a standalone vector element to `git`, an input formatted as a command-line flag (such as `-output=/user/../user/foo`) causes the `git` binary to interpret the parameter as an operational option rather than a positional revision range. Under this flag, `git log` redirects its formatted output directly to the specified file path on disk.

TODO [Pierre: Show the actual triggering input produced by ToFU, preferably exactly as recorded by the fuzzer after minimization. If the example shown here is the actual minimized input, state that explicitly. If it is only an illustrative input, replace it with or separately show the real input. A case study should distinguish the discovered proof of concept from an explanatory example.]

TODO [Pierre: Explain how ToFU's runtime oracle detects this behavior. For example, if the introspection layer observes the file creation or the subprocess arguments, state exactly what event is intercepted and what rule causes ToFU to flag it. The current text explains why the behavior is vulnerable, but not how ToFU automatically recognizes the violation.]

TODO [Pierre: Also explain whether ToFU recognizes the dangerous command-line option before execution or detects its effect afterward through runtime introspection. This distinction matters for understanding the architecture and should match the Design section.]

Furthermore, the ternary check on line 3 (`range_spec if ".." in range_spec else f"{range_spec}..HEAD"`) is subverted by directory traversal sequences: providing path traversal dots (`..`) in the file path satisfies the substring condition, thereby bypassing the automatic appending of `..HEAD` and passing the adversarial `-output` argument verbatim to the subprocess on line 6.

TODO [Pierre: There appears to be a LaTeX or code-formatting issue in the inline expression in this sentence. Check the nested texttt commands and braces carefully. I am not changing it here because the existing text should remain untouched, but this line should be fixed before submission.]

TODO [Pierre: Clarify that the two dots are useful here because the implementation checks only whether the string contains two consecutive dots. The security problem is not conventional path traversal by itself. The dots also manipulate the program's branch condition so that the injected Git option is passed unchanged. That interaction is an interesting semantic aspect of the bug and is worth emphasizing.]

TODO [Pierre: If ToFU generated or mutated the two-dot sequence automatically, explain that transition here. Showing how the fuzzer reaches the final option while satisfying the program's range check would make this a much stronger fuzzing case study.]

Although this vulnerability does not provide direct arbitrary code execution, it presents serious security and operational risks that can be exploited by an attacker in two distinct ways:
**Denial of Service through Storage and Inode Exhaustion (CWE-400):** By repeatedly invoking the tool with distinct file names or deeply nested paths across the filesystem, an attacker can generate a large volume of output files. This continuous file generation can quickly saturate available disk space or exhaust filesystem inodes, ultimately causing denial of service for the MCP server and co-located services.
**Arbitrary File Overwrite and Asset Integrity Corruption (CWE-732):** Depending on the filesystem permissions granted to the MCP server process, an attacker can overwrite existing critical files, such as configuration files, local database snapshots, or application assets. The overwritten files will contain the formatted `git log` commit history (`-pretty=%B`) rather than their original contents. Consequently, any subsequent process or service attempting to read or parse these corrupted files will encounter unexpected structures and fail, inducing silent runtime errors, degraded application behavior, or complete subsystem crashes.

TODO [Pierre: Recheck the CWE classification for the arbitrary-file-overwrite consequence. CWE-732 is Incorrect Permission Assignment for Critical Resource, which does not appear to describe the root cause here because the vulnerability does not assign incorrect file permissions. CWE-88 already captures the argument-injection root cause. If you want a secondary CWE for the resulting filesystem behavior, select one that actually corresponds to uncontrolled file path or file modification rather than permission assignment.]

TODO [Pierre: Be careful with the claim about deeply nested paths. State only behavior that was actually validated. Git can write to a supplied output path, but this does not necessarily imply that it creates arbitrary missing parent directories. The denial-of-service argument does not need this stronger claim because repeatedly creating distinct files in writable existing directories is sufficient to explain storage or inode exhaustion.]

TODO [Pierre: Close the case study with a short paragraph explaining why this bug illustrates ToFU's broader value. The key observation is that the parameter is type-correct, the subprocess invocation avoids shell interpretation, and ordinary malformed-input checking may therefore see nothing unusual. The vulnerability appears only when the string is interpreted according to Git's command-line semantics and compared against the intended meaning of range_spec. Connect this explicitly back to the LLM-to-tool trust boundary and semantic

sanitizer design.]

## 11 Discussion

**Multi-Agent Systems.** TOFU currently targets tool invocations made through an LLM orchestration framework. However, emerging multi-agent systems may distribute tool use across multiple interacting agents. For example, one agent may invoke a tool and pass its result to another agent, which subsequently uses the result as an argument to a different tool. Supporting such systems requires extending TOFU from individual tool invocations to multi-agent execution paths. One possible approach is to construct a harness over the agent communication graph and fuzz both agent-to-agent messages and the tool invocations reachable from each agent. The tool-specific sanitizers generated by TOFU can remain unchanged, but must be applied at each tool boundary reached during the execution. Furthermore, testing multi-agent executions requires maintaining state across multiple agent and tool invocations. We leave support for multi-agent systems to future work.

**Coverage of Third-Party Components.** Many tools are lightweight wrappers around external libraries, services, or applications. For example, a browser tool may contain only a small amount of orchestration code while most of its functionality executes inside Chromium. In such cases, coverage collected only from the tool implementation does not capture how extensively the underlying component is exercised. TOFU can be extended with component-specific coverage collectors that combine coverage from the tool implementation and its dependencies. For example, browser tools could additionally collect Chromium coverage, while tools invoking native libraries could collect coverage from instrumented library code. TOFU can then use the combined coverage when deciding whether a generated input exposes new behavior. Supporting arbitrary third-party components requires component-specific instrumentation interfaces, which we leave for future work.

**Stateful Tool Interactions.** TOFU currently fuzzes tools by generating inputs for individual tool interfaces. However, some tool behavior depends on state established by previous invocations. For example, one tool invocation may create a resource that is subsequently modified, queried, or deleted by another invocation. Bugs involving such state may therefore require a sequence of otherwise valid tool calls. TOFU can be extended to generate multi-call harnesses by identifying relationships between tool arguments and outputs, such as a resource identifier returned by one tool and consumed by another. The fuzzer can then mutate both the arguments and ordering of calls while applying the existing semantic sanitizers at each tool boundary. Determining useful invocation sequences across large tool collections remains an open problem, and we leave stateful tool fuzzing to future work.

## 12 Related Work

**Attacks on tool augmented LLMs.** Prior work shows that a value reaching a tool cannot be assumed to reflect benign user intent. Beyond direct adversarial manipulation of model outputs [16], Greshake et al. [7] demonstrate that content retrieved by an LLM-integrated application can smuggle in instructions that hijack agent behavior, establishing indirect prompt injection as a practical threat to tool-using agents. ToolHijacker [14] extends this to tool *selection*: an attacker who registers a tool with a description mimicking a legitimate one can intercept requests during retrieval, without touching the model itself. Les Dissonances [10] further shows that one tool's output can contaminate another tool's arguments, meaning even a fully trusted user prompt does not bound what a tool ultimately receives. Collectively, these results explain *how* an adversarial value arrives at a tool, motivating the threat model in Section 3. TOFU instead asks what happens *after* arrival: whether the implementation actually enforces the security properties its interface claims. Because this question is independent of delivery mechanism, we evaluate reachability through the normal tool-calling path separately.

**Automated vulnerability discovery.** The systems closest to TOFU share a common limitation: each starts from a fixed, predetermined set of weaknesses or attack patterns. Hasan et al. [8] apply general-purpose and MCP-specific vulnerability checks to published MCP servers, and MCP-SecLint [13] traces arguments toward a predefined list of security-sensitive sinks—both are static analyses that never execute the target, so a flagged pattern is never confirmed reachable via the tool's actual invocation path. MCP-Scanner [1] does execute tools, but only checks for a catalog of known MCP threats (tool poisoning, prompt injection), so execution buys confidence without expanding what can be detected. Chord [10] also executes real tools, using an oracle purpose-built for cross-tool harvesting and pollution; that oracle is precise for its narrow class but doesn't generalize to arbitrary argument-level properties like filesystem containment or destination restriction. As Table 2 summarizes, none of these systems derives its checks from the semantics a tool documents for its own arguments—so a semantically invalid path, network destination, or application-specific value stays invisible unless it happens to match an anticipated pattern or sink. TOFU inverts this: it starts from each argument's documented meaning, derives the security properties that meaning implies, and tests them directly—letting it surface weaknesses no one enumerated in advance.

**Structure-aware fuzzing and the oracle problem.** TOFU builds on conventional fuzzing infrastructure, using Atheris [5] to mutate structured, schema-valid invocations so that adversarial values remain schema-valid and reach the implementation. Validity alone, however, does not establish

that an accepted input was handled *safely*. Classical fuzzers rely on application-independent signals such as crashes or memory-safety violations, which miss tools that terminate normally while violating a documented property.

**LLM-assisted test generation.** A separate line of work like TitanFuzz [4] and Fizz4All [15] uses LLMs to test conventional softwares. On the other hand, in TOFU, the targets *are* LLM tools; where they differ is in what role the model plays. Approaches that ask an LLM to judge whether code or an execution is vulnerable inherit the model's hallucinations and nondeterminism straight into the verdict.

## 13 Conclusion

We presented TOFU, a tool-aware framework for identifying security and correctness bugs at the LLM-to-tool trust boundary. TOFU derives tool-specific semantic checks from existing tool metadata and a CWE-guided sanitizer corpus, then uses these checks to automatically generate fuzzing harnesses and semantic oracles. Across 569 tools from 11 repositories, TOFU identified 14 previously unknown bugs, including arbitrary file writing, MCP server crashes, incorrect tool descriptions, and denial-of-service vulnerabilities. These findings show that tool implementations frequently fail to enforce the semantics exposed to LLM orchestration frameworks, allowing malformed or unexpected model-generated arguments to reach security-sensitive operations. More broadly, our results demonstrate that tool metadata can serve not only as an interface for LLM agents, but also as a useful specification for systematically testing the security of the underlying tool ecosystem.

## References


[1] Parya Abadeh, Martin Lochner, Taha Ansari, and Fattane Zarrinkalam. MCP-Scanner: Detecting security risks in model context protocol systems. In *Proceedings of the 2026 ACM/IEEE 7th International Workshop on Engineering and Cybersecurity of Critical Systems (EnCyCriS)*, pages 41–48, 2026.

[2] Amazon Web Services. Strands agents: Agent loop. https://strandsagents.com/docs/user-guide/concepts/agents/agent-loop/, 2026.

[3] CrewAI. Crewai documentation. https://docs.crewai.com/, 2026.

[4] Yinlin Deng, Chunqiu Steven Xia, Haoran Peng, Chenyuan Yang, and Lingming Zhang. Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models. In *Proceedings of the 32nd ACM SIGSOFT international symposium on software testing and analysis*, pages 423–435, 2023.

[5] Google. Atheris: A coverage-guided, native python fuzzer, 2020.

[6] Google. Agent development kit 2.0. https://adk.dev/graphs/, 2026.

[7] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*, pages 79–90, 2023.

[8] Mohammed Mehedi Hasan, Hao Li, Emad Fallahzadeh, Gopi Krishnan Rajbahadur, Bram Adams, and Ahmed E. Hassan. Model context protocol (MCP) at first glance: Studying the security and maintainability of MCP servers. *ACM Transactions on Software Engineering and Methodology*, 2026.

[9] LangChain. Langgraph: Graph api. https://docs.langchain.com/oss/python/langgraph/graph-api, 2026.

[10] Zichuan Li, Jian Cui, Xiaojing Liao, and Luyi Xing. Les dissonances: Cross-tool harvesting and polluting in pool-of-tools empowered LLM agents. In *Network and Distributed System Security Symposium (NDSS)*, 2026.

[11] Microsoft. Autogen core. https://microsoft.github.io/autogen/stable/user-guide/core-user-guide/, 2026.

[12] Microsoft. Microsoft agent framework: Workflow builder and execution. https://learn.microsoft.com/en-us/agent-framework/concepts/workflows/builder-and-execution, 2026.

[13] Alfonso Pedro Ridao, Melissa Safari, Zachary Kang, and Nicola Dragoni. MCP-SecLint: An open-source static analyzer for detecting vulnerabilities in LLM tool integrations. In *Proceedings of the 12th ACM International Workshop on Security and Privacy Analytics (IWSPA)*, pages 89–100, 2026.

[14] Jiawen Shi, Zenghui Yuan, Guiyao Tie, Pan Zhou, Neil Zhenqiang Gong, and Lichao Sun. Prompt injection attack to tool selection in LLM agents. In *Proceedings of the Network and Distributed System Security Symposium*, 2026.

[15] Chunqiu Steven Xia, Matteo Paltenghi, Jia Le Tian, Michael Pradel, and Lingming Zhang. Fuzz4all: Universal fuzzing with large language models. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, pages 1–13, 2024.

[16] Andy Zou, Zifan Wang, Nicholas Carlini, Milad Nasr, J. Zico Kolter, and Matt Fredrikson. Universal and transferable adversarial attacks on aligned language models, 2023.

# A Appendix

TODO [This description is useless, we need to know how the sanitizer works.]

Table 7: List of used CWE-based sanitizers

| **Sanitizer** | **CWE** |
|---|---|
| External Configuration Control | CWE-15 |
| Path Traversal / Directory Traversal | CWE-22 |
| Sensitive File Access | CWE-22 / CWE-552 |
| Symlink Following | CWE-59 |
| Command Injection | CWE-77 / CWE-78 |
| CLI Argument Injection | CWE-88 |
| SQL Injection | CWE-89 |
| Dynamic Code Injection | CWE-94 |
| Log Injection / CRLF & ANSI Forging | CWE-117 |
| Memory Access Violation / Buffer Overflow | CWE-120 / CWE-787 / CWE-125 |
| Format String Injection | CWE-134 |
| Sensitive Information Leak in Error Messages | CWE-209 |
| Unnecessary Root Privileges / Privilege Escalation | CWE-250 / CWE-272 |
| Insecure Default File Permissions | CWE-276 / CWE-279 |
| Improper SSL/TLS Certificate Validation | CWE-295 |
| Cleartext Storage of Sensitive Information | CWE-312 / CWE-538 |
| Cleartext Transmission of Sensitive Information | CWE-319 |
| Cryptographic Nonce / IV Reuse | CWE-323 |
| Weak Hash Algorithm Usage | CWE-328 |
| Cryptographically Weak PRNG | CWE-338 |
| Divide By Zero | CWE-369 |
| Insecure Temporary File Permissions | CWE-378 / CWE-379 |
| Resource Exhaustion / Execution Hang | CWE-400 |
| File Descriptor Leak | CWE-403 / CWE-910 |
| Data Amplification / Decompression Bomb | CWE-409 |
| Unrestricted Dangerous File Upload | CWE-434 |
| Non-Exit on Failed Initialization | CWE-455 |
| Incomplete Resource Cleanup | CWE-459 |
| Cross-Session Data Leak | CWE-488 |
| Unverified Code Download | CWE-494 |
| Untrusted Deserialization | CWE-502 |
| Temporary File Leak | CWE-533 |
| Unchecked Loop Condition / CPU Exhaustion | CWE-606 |
| XML External Entity (XXE) & Entity Expansion | CWE-611 / CWE-776 |
| Reachable Assertion DoS | CWE-617 |
| Sensitive Data Logging | CWE-779 |
| Hardcoded Credentials | CWE-798 |
| Deadlock / Thread Lock Hang | CWE-833 |
| Type Confusion / Incompatible Type Access | CWE-843 |
| Prototype Pollution / Object Attribute Injection | CWE-915 |
| Server-Side Request Forgery (SSRF) | CWE-918 |
| CSV / TSV Formula Injection | CWE-1236 |
| Blocking Call in Asynchronous Event Loop | CWE-1322 |
| Unrestricted IP Address Binding | CWE-1327 |
| Regular Expression Denial of Service (ReDoS) | CWE-1333 |